🛡️ DevSecOps 2026–2030 Masterclass

🛡️ **Motto:** Zero to Industry-Grade Elite
Delivery Style: 5% Logic, 95% Hands-on
Outcome: A portfolio of security-engineered pipelines, cloud-native hardening labs, and compliance-ready evidence.

Curriculum Overview

Phase	Focus	Core Outputs (Assessable)	Primary Toolchain
PHASE 1	Foundations & Mindset (Shift-Smart)	Lab baseline + personal roadmap	Docker, VS Code, GitHub Actions
PHASE 2	Secure Design & AI-Powered Planning	Threat model + secure architecture (Fintech)	OWASP Threat Dragon, Jira
PHASE 3	Secure Coding (AI as your Peer)	Pre-commit policy + secrets workflow automation	SonarLint, Snyk, Gitleaks, Vault
PHASE 4	The Hardcore Pipeline (Build & Test)	Capstone: secure CI/CD with SAST/SCA/SBOM/DAST	SonarQube, Dependency-Check, Syft, Grype, ZAP
PHASE 5	Cloud-Native & Container Security	Hardened cluster + policy enforcement	Terraform, Ansible, Checkov, Trivy, Kyverno/OPA
PHASE 6	Runtime Observability & Vulnerability Mgmt	Runtime detection + SOC dashboards + vulnerability intake	Falco, ELK, Grafana, DefectDojo

Phase	Focus	Core Outputs (Assessable)	Primary Toolchain
PHASE 7	Advanced Frontiers (2030 Ready)	LLM AppSec demo + compliance automation + IR simulation	OWASP Top 10 for LLMs, Cloud Custodian

---

Academic Roadmap (Phases → Videos)

PHASE 1 — FOUNDATIONS & MINDSET (Shift-Smart)

▼ Video 1 — DevSecOps 2026: Evolution from Shift-Left to Shift-Smart

- **Theory:** Why “checkbox DevSecOps” is becoming obsolete
- **Theory:** The role of AI-driven automation and policy enforcement at scale
- **Exit Criteria:** You can articulate a modern SDLC security operating model (design → code → pipeline → runtime)

▼ Video 2 — The Ultimate Career Roadmap (2026–2030)

- **Practical:** Build a professional lab (Docker, VS Code, GitHub Actions)
- **Strategy:** High-impact skills for $100k+ global roles (portfolio-first, proof-driven)
- **Exit Criteria:** A baseline repo + workstation setup + roadmap milestones with measurable outputs

🔥 **Industry-Grade Tip:** Do not “learn tools.” Learn controls. Each tool is just an implementation of a control (SAST, SCA, SBOM, DAST, policy, detection).

PHASE 2 — SECURE DESIGN & AI-POWERED PLANNING

▼ Video 3 — Advanced AI-Powered Threat Modeling

- **Practical:** Model threats using OWASP Threat Dragon
- **Practical:** Use AI agents to enumerate attack vectors, abuse cases, and trust boundaries

- **Exit Criteria:** A threat model with prioritized mitigations mapped to backlog items

▼ Video 4 — Security Requirements & Governance

- **Practical:** Jira integration for security requirements tracking
- **Practical:** Define security gates (Build Fail/Pass) as objective criteria
- **Exit Criteria:** A “Security Gates” policy that can be enforced in CI/CD

▼ Video 5 — [MAJOR PROJECT] Secure Architecture for a Fintech Application

- **Deliverable:** Secure architecture (diagrams + controls + data flows)
- **Focus:** Authentication/authorization, secrets, logging, encryption, tenancy boundaries
- **Exit Criteria:** Architecture decisions are threat-model-driven and implementation-ready

🔒 **Industry-Grade Tip:** Architect for evidence. Every control should have an auditable signal (pipeline report, config artifact, runtime alert, ticket trail).

PHASE 3 — SECURE CODING (AI as your Peer)

▼ Video 6 — IDE Defense: Implementing Pre-Commit Hooks

- **Practical:** Pre-commit hooks as “developer-local security gates”
- **Practical:** Configure SonarLint + Snyk in VS Code
- **Exit Criteria:** Pull requests cannot land without baseline code quality + security checks

▼ Video 7 — AI-Assisted Secure Development

- **Practical:** Use GitHub Copilot for secure coding workflows
- **Practical:** Identify and correct AI-generated security flaws (unsafe deserialization, injection, weak crypto, auth bypass patterns)

- **Exit Criteria:** You can review AI output like a security engineer, not like an autocomplete user

▼ Video 8 — Secrets Management: The Zero Trust Framework

- **Practical:** Automate secret scanning with Gitleaks
- **Practical:** Dynamic secrets + centralized policy using HashiCorp Vault
- **Exit Criteria:** No hardcoded secrets; rotation + least privilege are default

▼ Video 9 — [MINI PROJECT] Automating Secret Scanning in GitHub Workflows

- **Deliverable:** GitHub Actions workflow with enforced secret scanning gates
- **Exit Criteria:** Leaked secrets fail builds and produce actionable developer feedback

> 📖 **Industry-Grade Tip:** "No secrets in Git" is not a policy statement; it is an enforced control with automated detection, prevention, and rotation.

PHASE 4 — THE HARDCORE PIPELINE (Build & Test)

▼ Video 10 — SonarQube Mastery (Enterprise-Grade SAST)

- **Practical:** Deploy SonarQube Community Edition
- **Practical:** Enforce Quality Gates (fail builds on high severity issues)

▼ Video 11 — Software Supply Chain & SCA

- **Practical:** Identify vulnerable libraries via Snyk + OWASP Dependency-Check
- **Exit Criteria:** Dependencies are governed (policy + approvals) not "best effort"

▼ Video 12 — SBOM (Software Bill of Materials): The 2026 Standard

- **Practical:** Generate + scan SBOMs with Syft and Grype
- **Exit Criteria:** Every release artifact ships with SBOM evidence

▼ Video 13 — AI-Driven DAST with OWASP ZAP

- **Practical:** Automated web vulnerability scans inside CI/CD
- **Exit Criteria:** DAST findings are triaged with severity + reproducibility

▼ Video 14 — [CAPSTONE PROJECT] Engineering the Ultimate Secure CI/CD Pipeline

- **Deliverable:** A production-grade CI/CD pipeline integrating SAST + SCA + SBOM + DAST with gated promotion
- **Exit Criteria:** A release cannot be promoted without passing defined security gates and producing evidence artifacts

🔒 **Industry-Grade Tip:** Security gates must be deterministic. If developers cannot reproduce a failure locally or in CI, the gate becomes noise and will be bypassed.

PHASE 5 — CLOUD-NATIVE & CONTAINER SECURITY

▼ Video 15 — IaC Security (Terraform & Ansible)

- **Practical:** Scan infrastructure code with Checkov + Terrascan
- **Exit Criteria:** Misconfigurations are prevented pre-deploy, not discovered post-incident

▼ Video 16 — Container Hardening & Distroless Images

- **Practical:** Trivy scanning + attack surface reduction using Distroless images
- **Exit Criteria:** Images are minimal, signed where possible, and scanned on build + deploy

▼ Video 17 — Kubernetes Security (Policy-as-Code)

- **Practical:** Enforce cluster policy via Kyverno or OPA
- **Exit Criteria:** Workloads must comply with baseline controls (non-root, resource limits, network policy readiness)

▼ Video 18 — [MINI PROJECT] Deploying a Hardened Kubernetes Cluster on AWS/Azure

- **Deliverable:** Hardened cluster + baseline policies + scanning pipeline integration
- **Exit Criteria:** Cluster posture is measurable and enforceable

🚀 **Industry-Grade Tip:** Cloud security maturity = policy + automation. Manual “review checklists” do not scale beyond small teams.

PHASE 6 — RUNTIME OBSERVABILITY & VULNERABILITY MANAGEMENT

▼ Video 19 — Runtime Threat Detection with Falco

- **Practical:** Detect and mitigate live attack behavior in real time
- **Exit Criteria:** You can distinguish false positives from true suspicious activity via rules + context

▼ Video 20 — Security Observability (The SOC Ecosystem)

- **Practical:** Security dashboards with ELK Stack + Grafana
- **Exit Criteria:** Security metrics are decision-grade (coverage, MTTR, trendline, risk drivers)

▼ Video 21 — Vulnerability Management with DefectDojo

- **Practical:** Aggregate tool data into DefectDojo as a single pane of glass
- **Exit Criteria:** Findings lifecycle is operational (intake → triage → assign → verify → close)

⚙️ **Industry-Grade Tip:** Observability is not dashboards. It is the ability to answer: “What changed, what broke, what’s exploitable, and what do we do next?”

PHASE 7 — ADVANCED FRONTIERS (2030 Ready)

▼ Video 22 — AI/LLM Application Security

- **Practical:** OWASP Top 10 for LLMs demo: prompt injection, data leakage, insecure plugin/tool use
- **Exit Criteria:** You can threat model LLM apps (RAG, tools, agents) and propose measurable mitigations

▼ Video 23 — Compliance Automation (SOC2 / ISO 27001)

- **Practical:** Automated compliance auditing with Cloud Custodian
- **Exit Criteria:** Controls are continuously evaluated and produce evidence artifacts

▼ Video 24 — Real-World Incident Response Simulation

- **Practical:** Pipeline remediation + professional post-mortem analysis
- **Exit Criteria:** You can run a blameless incident lifecycle with concrete corrective actions

▼ Video 25 — Global Career Strategy & Interview Excellence

- **Logic:** Portfolio building, scenario-based Q&A, and navigating the 2030 job market
- **Exit Criteria:** A hiring-ready portfolio narrative mapped to modern DevSecOps outcomes

★ **Industry-Grade Tip:** Your differentiator is operational credibility: you can design controls, automate enforcement, and defend decisions with evidence.